

Reviewer Pack — Coxeter Group Enumeration and Circuit Lower Bounds

Entry 6c01cc578f5d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 02:13:37 UTC

Coxeter Group Enumeration and Circuit Lower Bounds

Entry ID: 6c01cc578f5d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 02:13:37 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Group Theory (Coxeter Groups) **Field B** (complexity object): Complexity Theory (Boolean Circuit Complexity)

Statement:

[‘For every Boolean circuit C with n inputs and size s , the number of distinct reduced words for its computation using the action of a Coxeter group G is at least $\tilde{O}(s^{1/3})$.’, ‘Equivalently, there exists an absolute constant $c > 0$ such that for all circuits C with n inputs, if C can be computed by the action of a Coxeter group with s reduced words, then $|V(C)| \leq cs^{1/3}$.’, ‘Here, $|V(C)|$ denotes the number of vertices in the vertex cover of the circuit C .’]

Rationale (proposer’s reasoning):

[‘Coxeter groups have been used to study the complexity of algorithms in various contexts. This conjecture explores their potential in bounding the size of Boolean circuits.’, “The action of a Coxeter group on a set can be represented as a sequence of operations that preserve the group’s properties, potentially leading to non-trivial lower bounds on circuit size.”, ‘This could provide new insights into the structure of Boolean circuits and their complexity.’]

Taxonomy category: COMBINATORIAL_GROUP_THEORY_CIRCUIT_COMPLEXITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f177553067afe098

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Boolean circuit C with n inputs and size $s \leq 40$, if $|V(C)| \leq cs^{1/3}$ for some constant $c > 0$ and all generated circuits, then the conjecture is supported; otherwise, it is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n, s):
        if n == 1:
            return [('AND', [0])]
        else:
            left = generate_circuit(n // 2, s // 2)
            right = generate_circuit(n - n // 2, s - s // 2)
            return [('OR', [left, right])]

    def vertex_cover(circuit):
        if not circuit:
            return 0
        gate, inputs = circuit[0]
        if gate == 'AND':
            return 1 + max(vertex_cover(inputs[0]), vertex_cover(inputs[1]))
        elif gate == 'OR':
            return 1 + min(vertex_cover(inputs[0]), vertex_cover(inputs[1]))

    def count_reduced_words(circuit):
        if not circuit:
            return 1
        gate, inputs = circuit[0]
        if gate == 'AND':
            return count_reduced_words(inputs[0]) * count_reduced_words(inputs[1])
        elif gate == 'OR':
            return count_reduced_words(inputs[0]) + count_reduced_words(inputs[1])
```

```

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    s_values = [min(s, n * (n - 1) // 2) for s in range(1, n * (n - 1) // 2 + 1)]
    for s in s_values:
        circuit = generate_circuit(n, s)
        num_vertices = vertex_cover(circuit)
        num_reduced_words = count_reduced_words(circuit)
        results.append({
            "metric_name": "vertex_cover_size",
            "metric_value": num_vertices,
            "instances_tested": 1,
            "conjecture_holds": num_vertices <= s ** (1/3),
            "counterexample": ""
        })

mean_value = sum(result["metric_value"] for result in results) / len(results)
std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

return {
    "seed": seed,
    "mean_vertex_cover_size": mean_value,
    "std_vertex_cover_size": std_value,
    "support_fraction": support_fraction
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["mean_vertex_cover_size"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["mean_vertex_cover_size"] - mean_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["support_fraction"] == 1) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["support_fraction"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["support_fraction"])
        counterexample = "vertex_cover_size"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63a1b0f7.py", line 81, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63a1b0f7.py", line 54, in run_trial
    num_vertices = vertex_cover(circuit)
                  ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63a1b0f7.py", line 36, in vertex_cover
    return 1 + min(vertex_cover(inputs[0]), vertex_cover(inputs[1]))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63a1b0f7.py", line 36, in vertex_cover
    return 1 + min(vertex_cover(inputs[0]), vertex_cover(inputs[1]))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_63a1b0f7.py", line 34, in vertex_cover
    return 1 + max(vertex_cover(inputs[0]), vertex_cover(inputs[1]))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13084
2	preregistration	ollama_remote	glm4:latest	0	0	5404
3	novelty	ollama_remote	glm4:latest	0	0	4856
4	novelty	ollama_remote	glm4:latest	0	0	4772
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20160
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9717
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9616
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10457
9	judge	ollama_remote	glm4:latest	0	0	8335

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 86400 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/6c01cc578f5d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/6c01cc578f5d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/6c01cc578f5d.tar.gz (if generated)